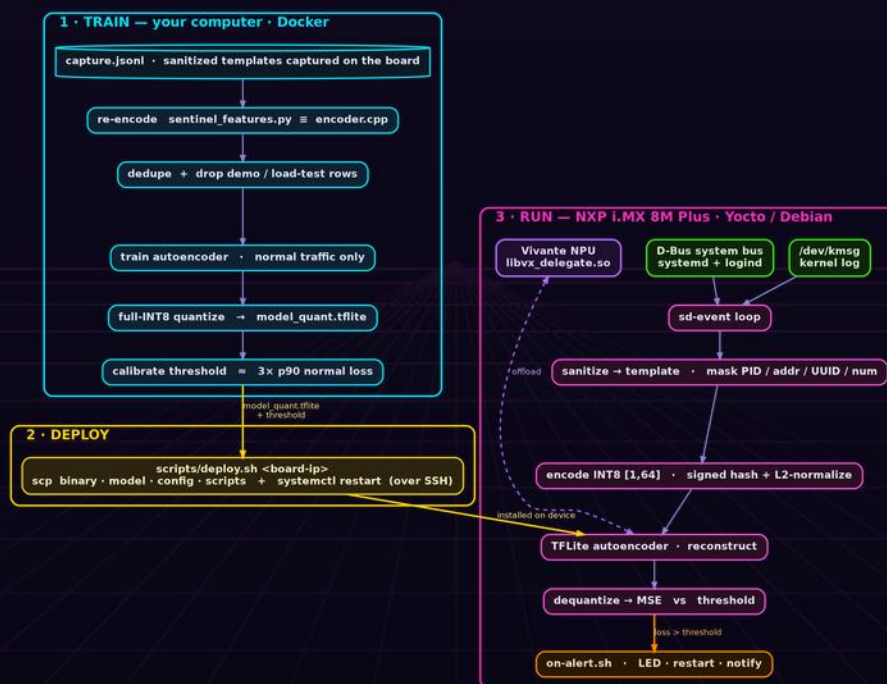


Why standard log parsing fails on embedded boards

...and why a custom on-device model fixes it

Embedded boards emit a relentless stream of kernel and system logs. The usual answers - regex/keyword rules or shipping every line to a cloud stack - are brittle, noisy, network-hungry, and blind to the failures nobody wrote a rule for yet. This brief explains why, and makes the case for a tiny anomaly-detection model that learns what 'normal' looks like for your specific device and runs entirely on its NPU.



the sentinel-imx-sys pipeline: train on host -> deploy over SSH -> detect on the NPU

Standard log parsing does not fit the edge

Two approaches dominate today, and both struggle on a constrained, remote, device-specific board:

- **Rules only catch what you already predicted.**

Regex and keyword alerts fire only for failure modes someone anticipated and hand-coded. Embedded systems fail in device- and firmware-specific ways, so the unknown-unknowns - the incidents you most want to catch - sail straight through.

- **Rules rot with every update.**

Each BSP, kernel, or driver bump changes log wording and format. Rule sets silently break and demand endless re-tuning and maintenance that does not scale across fleets or releases.

- **Alert fatigue drowns the real signal.**

Threshold and keyword rules light up on benign chatter. Operators mute noisy alerts - and the one message that mattered is muted with them.

- **'Normal' is device-specific; static rules are not.**

A line that is perfectly routine on one unit is a red flag on another. One-size-fits-all rules cannot encode a per-device notion of healthy behavior.

- **The cloud is the wrong place to decide.**

Shipping logs to ELK/Loki assumes bandwidth, connectivity, and power the edge often lacks, and it adds latency, running cost, and data sovereignty / privacy problems. A board cannot react in real time if the verdict lives in a datacenter.

- **Heavy agents, blind to novelty.**

Full log-processing agents (Logstash, Fluentd) are large for a board's CPU, RAM, and flash budget - and still never flag a genuinely new fault or a slow degradation until a human writes a rule after the outage.

Learn 'normal' on the device, flag the rest

Instead of enumerating everything that can go wrong, train a small autoencoder on the board's own normal logs. It reconstructs familiar patterns well and unfamiliar ones poorly - so a high reconstruction error flags novelty you never had to anticipate. It runs entirely on the Vivante NPU.

- **No rules to write - it learns from real traffic.**

A capture-first workflow records the device's own logs; you train from them. The model encodes what healthy looks like for this exact unit.

- **Catches the unknown.**

Reconstruction error surfaces novel anomalies and slow drift, not just pre-listed keywords.

- **Fully on-device and offline.**

No cloud, no log shipping: data stays local (privacy + sovereignty) and the board reacts in real time.

- **Tiny footprint.**

An INT8-quantized model on the NPU costs under 1% CPU and about 35 MB RSS, leaving the CPU free for the real workload.

- **Low maintenance.**

After a BSP or kernel change, re-capture a fresh baseline and retrain - no rule archaeology.

- **Noise-resistant by design, and actionable.**

Log templating plus L2-normalized feature hashing score events by pattern novelty (not line length or PID churn), and each anomaly runs your own script: LED, service restart, or notify.

Rule-based / cloud vs on-device model

Dimension	Rule-based / cloud parsing	On-device model (sentinel-imx)
Unknown failures	● Missed - known patterns only	● Detected via reconstruction error
Maintenance	● Rewrite rules every update	● Recapture + retrain a baseline
Connectivity	● Needs network / cloud	● Fully offline, on-device
Time to act	● Round-trip to a datacenter	● Real-time, local
Data privacy	● Logs leave the device	● Data stays on the device
Footprint	● Heavy log agents	● INT8 model, <1% CPU, ~35 MB
Notion of normal	● One static rule set	● Learned per device

How sentinel-imx-sys delivers it

The daemon listens to /dev/kmsg and the D-Bus system bus, sanitizes each line into a stable template, encodes it into an INT8 [1,64] vector, and scores it with the autoencoder on the Vivante NPU. You train on your computer inside Docker, deploy over SSH with one script, and detection then runs on-device - no cloud in the loop.

MEASURED ON THE BOARD

0

false positives - idle

213-325

anomaly loss (MSE)

150

calibrated threshold

~38%

NPU load - Vivante c1

35 MB

daemon RSS

<1%

CPU while parsing

The takeaway

You get proactive, private, low-cost health monitoring at the edge that improves as it sees more of the device's real life - catching the failures a rule set never anticipated, without sending a single log line off the board.